

TRIE-AWARE TOOL MEMORY · FYP RESEARCH REPORT

Task C & Task D

Dataset normalization, schema-token accounting, tool-access patterns, deterministic ordering, and analytical trie reuse.

● Local analysis complete · Cluster validation pending

45,815

CANONICAL TOOLS

9,201

BENCHMARK TASKS

70

MEDIAN SCHEMA
TOKENS

31.80%

BEST TOOLRET
ESTIMATE

18.88%

BEST BFCL ESTIMATE

CONTENTS

Dataset provenance

ToolRet

BFCL V4

Task C — Inspect and normalize tool datasets

Question from the research brief

Method and reasoning

Task C results

Task C answer

Task D — Analyze initial tool-access patterns

Question from the research brief

Method and reasoning

Task D results

Task D answer

Reproduction

Executive answer

Task C establishes a reproducible canonical tool corpus and token accounting pipeline. Task D finds that deterministic ordering improves analytical prefix sharing on multi-tool workloads: schema-cost weighting is strongest for ToolRet, while frequency ordering is strongest for the selected BFCL subset. GPU latency and correctness remain cluster measurements.

This document answers Task C and Task D from `initial-research-brief.md`. It records the reproducible methodology and reasoning used to reach the conclusions. It does not claim that the local analytical cache estimates are measured vLLM or GPU results.

Dataset provenance

All inputs are public research datasets. No SoC, PayPal, private MCP, or production request data was used.

ToolRet

Sources:

- ToolRet paper
- ToolRet tool corpus
- ToolRet query corpus

Downloaded inventory:

TOOL CORPUS CONFIGURATION	TOOLS
Web	37,292
Code	3,794
Customized	3,367
Total	44,453

The query corpus contains 7,961 tasks across 35 configurations. Each query has gold relevance labels identifying the tools considered relevant to that query. All 7,961 tasks had parsed labels, and every label resolved to a tool in the downloaded corpus.

BFCL V4

Source:

- Berkeley Function Calling Leaderboard data

Downloaded subset:

BFCL FILE/CATEGORY	TASKS
Simple Python	400
Multiple	200
Parallel	200
Parallel multiple	200
Irrelevance/no-tool	240
Total	1,240

The files contain 1,362 distinct canonical function definitions after 555 repeated identical definitions were merged.

Task C — Inspect and normalize tool datasets

Question from the research brief

For ToolRet and a manageable BFCL subset:

1. document the available fields and file formats;
2. parse tool names, descriptions, parameters, required fields, and examples;
3. implement one canonical serialization format;
4. measure token length per tool;
5. report missing or malformed schema fields; and
6. group tools by source, domain, or server where possible.

The required output is a reproducible script and a short data report.

Method and reasoning

1. Keep the two datasets conceptually separate

ToolRet and BFCL provide different evidence:

- ToolRet labels indicate **gold relevance** for retrieval.
- BFCL `function` arrays indicate **tools exposed in a task menu**.

BFCL menu exposure is not a successful model call, and neither source is a chronological production trace. Combining the frequencies without preserving this distinction would produce an ambiguous statistic, so the pipeline normalizes both datasets but analyzes their tasks separately.

2. Parse the source formats

ToolRet corpus files are Parquet records containing an `id` and a heterogeneous JSON-encoded `documentation` field. Query Parquet records contain an `id`, query/instruction text, JSON-encoded relevance `labels`, and category metadata.

BFCL files use JSON Lines despite the `.json` extension. Each task contains a conversation-like `question` and one or more inline function definitions in its `function` array.

The normalizer accepts common variants such as:

- `name`, `tool_name`, `function_name`, or `api_name`;
- `description`, `functionality`, or `summary`;
- `parameters`, `input_schema`, `doc_arguments`, `arguments`, or `api_arguments`; and
- `examples`, `example`, `example_code`, or `usage`.

Examples are preserved as normalized metadata. They are not inserted into the initial canonical prompt serialization because many records do not provide them consistently.

3. Use one deterministic serialization

Each tool is converted to an OpenAI-compatible function shape:

```
{
  "type": "function",
  "function": {
    "name": "...",
    "description": "...",
    "parameters": {
      "type": "object",
      "properties": {},
      "required": []
    }
  }
}
```

Object keys are sorted, unnecessary whitespace is removed, and UTF-8 text is preserved. Deterministic serialization is necessary because exact prefix caching depends on token identity and order, not merely semantic equivalence.

BFCL functions are assigned a stable identity using their function name and a hash of the canonical serialization. This merges repeated identical definitions while keeping different schemas separate.

4. Count canonical schema tokens

`schema_tokens` is the number of tokens in one canonical tool definition using the official `Qwen/Qwen3-0.6B` tokenizer:

```
schema_tokens(tool) =
  number of tokenizer tokens in canonical_json(tool)
```

Only the tokenizer is used locally. Qwen model weights are not downloaded and no model inference occurs. These counts exclude the final model chat template, tool-list separators, system message, and user message. The complete rendered prompt must therefore be re-tokenized on the cluster.

5. Preserve problems as explicit flags

The pipeline records missing or malformed fields instead of silently inventing rich schemas. Empty parameter objects are flagged for visibility but can be legitimate for no-argument functions.

Task C results

Combined inventory

DATASET	CANONICAL TOOLS	TASKS	TASK EVIDENCE
ToolRet	44,453	7,961	Gold relevance
BFCL subset	1,362	1,240	Exposed menu
Combined	45,815	9,201	Kept separate in analysis

Canonical schema-token distribution

STATISTIC	SCHEMA TOKENS
Minimum	28
P25	49
Median	70
Mean	92.08
P75	109
P90	169
P95	220
P99	357
Maximum	8,652

Most tools are relatively short, but the long tail is substantial. The largest canonical schema, `Crypto_Arbitrage_crypto_arb`, contains 8,652 tokenizer tokens. Large schemas can dominate prompt-prefill cost even when they occur less frequently.

Schema-quality flags

FLAG	TOOLS
Empty parameters	19,887
Missing description	994
Missing parameters	155
Malformed parameters	150
Missing name	2
Required field absent from properties	1

There are 19,921 tools with at least one flag. This number should not be read as 19,921 unusable tools: nearly all are flagged because they have an empty parameter object.

Source grouping

SOURCE	CANONICAL TOOLS
ToolRet web	37,292
ToolRet code	3,794
ToolRet customized	3,367
BFCL simple Python	220
BFCL multiple	480
BFCL parallel	153
BFCL parallel multiple	270
BFCL irrelevance	239

The BFCL source counts are distinct canonical definitions rather than the number of tasks.

Task C answer

Task C is complete for all ToolRet tools/queries and the five selected BFCL V4 categories.

The data can be parsed into one deterministic representation, and all ToolRet gold labels resolve correctly. Median canonical schema length is 70 Qwen tokens, but schema cost has a long tail reaching 8,652 tokens. The main data quality issue is widespread empty parameter objects; malformed and truly missing fields are much less common.

Reproducible artifacts:

- `scripts/download_datasets.py`
- `scripts/run_pipeline.py`
- `src/tatm/normalize.py`
- `src/tatm/serialization.py`
- `reports/dataset-inventory.md`

Task D — Analyze initial tool-access patterns

Question from the research brief

Using gold labels or successful trajectories where available, compute:

1. tool occurrence frequency;
2. schema-token-weighted frequency;
3. pair and triple co-occurrence;
4. conditional transitions between tools;
5. number of tools per task;
6. total schema tokens per task;
7. domain-level locality; and
8. trie size and compression under different orderings.

Compare original, alphabetical, fixed-seed random, frequency, schema-cost-weighted frequency, and FP-tree-style global order. Clearly separate benchmark statistics from generated replay statistics.

Method and reasoning

1. Define frequency according to the available evidence

For ToolRet:

```
frequency(tool) =  
    number of tasks whose gold relevance labels contain the tool
```

For BFCL:

```
frequency(tool) =  
    number of task menus that expose the tool
```

A tool is counted at most once per task. ToolRet frequency is a gold requirement/relevance signal. BFCL frequency is only an exposure signal. Neither is automatically production popularity.

2. Define a cost-aware score

The first approximation of reusable prefill value is:

```
schema_cost_weighted_score(tool) =  
    frequency(tool) × schema_tokens(tool)
```

This prioritizes a long, moderately frequent schema over a very short schema when the expected repeated prefill work is larger. It remains an approximation: actual value also depends on co-occurrence, prefix position, cache residency, block boundaries, eviction, and measured prefill latency.

3. Make every ordering deterministic

Only the already-selected tool set is reordered. Retrieval is not changed.

Frequency order:

```
sort key = (-frequency, tool_id)
```

Schema-cost-weighted order:

```
sort key = (  
    -(frequency × schema_tokens),  
    -frequency,  
    tool_id  
)
```

The stable `tool_id` resolves all ties. Fixed-seed random baselines use a stable SHA-256 rank of `seed:tool_id`, so rerunning the experiment produces the same order.

Classic FP-tree global order is descending global support. It is therefore intentionally identical to the frequency baseline in this first implementation. Conditional FP-tree mining would be a later, distinct method.

4. Measure co-occurrence and adjacency without overclaiming

Pair and triple co-occurrence count unordered combinations appearing in the same task.

Ordered adjacency is calculated from consecutive tool IDs in a task:

$$P(\text{next} = B \mid \text{current} = A) = \frac{\text{count}(A \text{ followed by } B)}{\text{all outgoing adjacency counts from } A}$$

ToolRet label order and BFCL menu order are not confirmed execution order. These statistics are therefore reported as ordered-adjacency proxies, not as successful-call transitions.

5. Compare trie sharing

For a workload of ordered tool sequences:

$$\begin{aligned} \text{naive_nodes} &= \text{sum}(\text{number of tools in every request}) \\ \text{trie_compression} &= \\ &1 - (\text{unique trie nodes} / \text{naive_nodes}) \end{aligned}$$

Before inserting each sequence, the analysis finds its longest prefix already present in the trie. Shared canonical schema tokens are rounded down to 16-token blocks:

$$\begin{aligned} \text{cacheable_tokens} &= \\ &\text{floor}(\text{shared_schema_tokens} / 16) \times 16 \\ \text{estimated_reuse} &= \\ &\text{total_cacheable_tokens} / \text{total_schema_tokens} \end{aligned}$$

This is a tool-level analytical estimate with an unbounded retained trie. It does not include the final chat template, physical vLLM cache eviction, GPU pressure, or scheduler behavior.

6. Test different workload assumptions

Four replays are compared:

- **Empirical:** public dataset file/configuration order.
- **Uniform:** uniform task sampling with replacement.
- **Skewed:** controlled sampling weighted by aggregate benchmark tool support.
- **Session-bursty:** tasks from the same domain grouped contiguously.

Uniform and skewed replays can repeat tasks. Session-bursty replay rearranges each task once. These are controlled systems workloads, not claims about production traffic.

Task D results

Tools and schema tokens per task

DATASET PARTITION	MEDIAN TOOLS	MEAN TOOLS	P95 TOOLS	MEDIAN SCHEMA TOKENS	P95 SCHEMA TOKENS
ToolRet gold	1	1.77	4	124	387
BFCL exposed	1	1.55	3	119	362.2

The median task contains only one tool in both datasets. Reordering cannot improve a one-tool sequence, so the useful regime is repeated multi-tool workflows.

Highest ToolRet gold-label frequencies

TOOL	FREQUENCY	SCHEMA TOKENS	FREQUENCY × TOKENS
Finish	350	33	11,550
finish	170	49	8,330
get_closing_parenthesis	70	46	3,220
stack_insert	70	49	3,430
stack_pop	70	44	3,080
account_login	64	76	4,864

The two **Finish** entries have different stable IDs and come from different sources, so they remain distinct tools.

Highest ToolRet schema-cost-weighted scores

TOOL	FREQUENCY	SCHEMA TOKENS	WEIGHTED SCORE
Agent.WalkTo	42	492	20,664
GET_search	10	1,349	13,490
Finish	350	33	11,550
Agent.Find	12	779	9,348
list_properties	8	1,090	8,720

This demonstrates why frequency and expected saved prefill work are not identical rankings.

Co-occurrence

ToolRet contains several strong repeated multi-tool patterns:

```
get_closing_parenthesis
→ stack_insert
→ stack_pop
```

This triple appears in 70 tasks.

```
create_object_dict
→ update_object_dict
→ get_final_object
```

This triple appears in 62 tasks.

BFCL co-occurrence is much weaker in the selected subset: its leading pairs and triples generally appear in only two tasks.

Empirical-order trie comparison

DATASET	ORDERING	TRIE-NODE COMPRESSION	ESTIMATED BLOCK-TOKEN REUSE
ToolRet	Original	29.45%	26.91%
ToolRet	Alphabetical	30.77%	26.86%
ToolRet	Random seed 7	29.74%	25.97%
ToolRet	Random seed 42	30.86%	26.70%
ToolRet	Random seed 101	29.75%	26.43%
ToolRet	Frequency	36.88%	31.35%
ToolRet	Schema-cost weighted	36.52%	31.80%
ToolRet	FP-tree global	36.88%	31.35%
BFCL	Original	11.89%	10.98%
BFCL	Alphabetical	16.64%	15.38%
BFCL	Random seed 7	16.54%	14.60%
BFCL	Random seed 42	16.22%	14.25%
BFCL	Random seed 101	16.17%	14.59%
BFCL	Frequency	21.07%	18.88%
BFCL	Schema-cost weighted	20.55%	18.76%
BFCL	FP-tree global	21.07%	18.88%

For ToolRet, schema-cost weighting improves the estimated reusable token ratio from 26.91% to 31.80%, a gain of 4.89 percentage points over original order.

For BFCL, frequency ordering improves the estimate from 10.98% to 18.88%, a gain of 7.90 percentage points over original menu order.

Sensitivity to replay assumptions

Best ordering within each replay:

DATASET	REPLAY	BEST ESTIMATED REUSE
ToolRet	Empirical	31.80%
ToolRet	Uniform with replacement	52.12%
ToolRet	Support-skewed	80.18%
ToolRet	Session-bursty	31.80%
BFCL	Empirical	18.88%
BFCL	Uniform with replacement	41.11%
BFCL	Support-skewed	61.39%
BFCL	Session-bursty	18.88%

The large replay differences show that repeated requests and popularity skew can matter more than the choice between two good ordering rules. They also show why a claimed cache benefit must state its traffic assumptions.

The empirical and session-bursty orders have very high same-domain adjacency because the public files are already grouped by configuration. This should not be interpreted as naturally occurring production session locality.

Task D answer

Task D is complete as a benchmark and controlled-replay analysis.

The data provides evidence that deterministic ordering can increase analytical prefix sharing, particularly for repeated multi-tool workflows:

- ToolRet performs best on reusable token volume with schema-cost-weighted ordering.
- BFCL performs best with frequency ordering in the selected subset.
- Frequency/FP-tree order creates the smallest ToolRet trie, while schema-cost weighting reuses slightly more token volume.
- ToolRet contains stronger repeated co-occurrence patterns than the selected BFCL subset.
- Most tasks contain only one tool, where reordering cannot help.
- Replay locality and repeated requests strongly affect the result.

The current recommended deterministic rules are:

ToolRet:

sort by $-(\text{frequency} \times \text{schema_tokens})$, $-\text{frequency}$, `tool_id`

BFCL:

sort by $-\text{frequency}$, `tool_id`

These are benchmark-derived rules, not production policies. With approved production traces, ToolRet gold frequency and BFCL menu frequency should be replaced or supplemented by successful-call frequency, measured prefill time, session locality, authorization, and cache residency.

The analysis does **not** yet prove a latency speedup. CUDA vLLM experiments must validate exact rendered token prefixes, cache hits, computed prompt tokens, prefill latency, TTFT, cache eviction, GPU memory, and BFCL correctness.

Reproducible artifacts:

- `src/tatm/analysis.py`
- `src/tatm/prompting.py`
- `scripts/build_cluster_workload.py`
- `reports/access-patterns.md`
- `reports/analysis-summary.json`
- `reports/tables/ordering-results.csv`

Reproduction

```
uv sync
uv run python scripts/download_datasets.py
uv run python scripts/run_pipeline.py
uv run pytest
```

Raw and normalized datasets are intentionally ignored by Git. The compact reports and source tables are stored under `reports/`.

